COM-480 · DATA VISUALIZATION · EPFL · PROCESS BOOK

Decanting the Web

An interactive data story about FineWeb, the dataset that feeds language models

Author: Alireza Abdollahpoorrostam · SCIPER 380830 **Dataset:** HuggingFace FineWeb · `sample-10BT` **Stack:** D3.js v7 · vanilla ES modules

Milestone: 3 (final)

01 The idea, and who it is for

Every large language model is what it eats. Before a model can write, it must read — and most of what it reads comes from web datasets like **FineWeb**. Yet for most people the training data of AI is a black box: enormous, abstract, invisible. Our goal was to make that black box *tangible* — to let a curious, non-expert reader *see* the web being distilled into the text a model learns from.

We chose a single, guided **scrollytelling narrative** rather than a dashboard. The dataset is rich, but our audience is not data engineers — it is the ML-curious public, students, and anyone who has wondered "where does ChatGPT's knowledge come from?". A linear story lets us build one insight on top of the next, while still allowing free exploration through controls inside each chart.

100T→15T	96	~340	0.57
tokens, raw → filtered	Common-Crawl snapshots	median tokens / doc	domain Gini

02 The dataset & the questions

FineWeb (Penedo et al., NeurIPS 2024) is a 15-trillion-token English corpus built by filtering Common Crawl. In Milestone 1 we streamed 200,000 documents of the public `sample-10BT` subset and produced a 24-section exploratory analysis. From that EDA, four questions emerged as the spine of our story:

How is the data made?
What does the curation pipeline actually remove, and how much survives at each step?

What does a document look like?
How long is a typical page, and how confidently English is it?

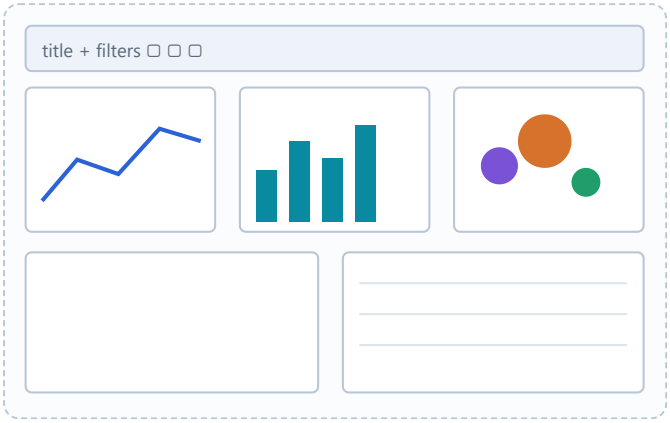
Where & when is it from?
How is the corpus spread across time and across the domains of the web?

How concentrated is it?
Do a few domains dominate, or is the web's text broadly distributed?

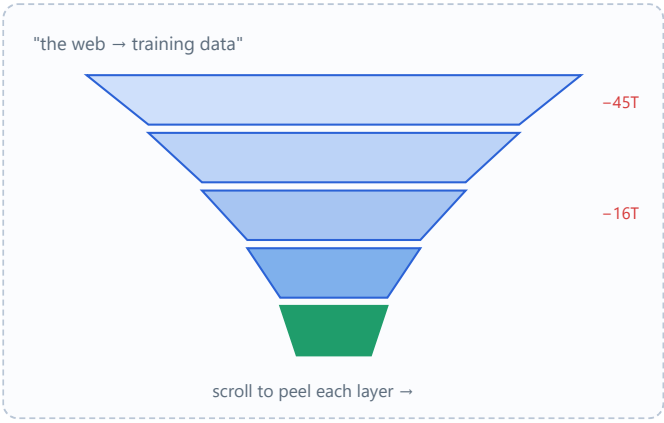
This process book describes how we went from those questions and our first paper sketches to the final interactive product — the design decisions, the dead-ends, and who did what.

03 From Milestone 1 sketches to the final design

Our Milestone-1 proposal contained two competing concepts. The first was a classic **multi-view dashboard** — every EDA figure on one screen. The second was a **guided story** built around the curation funnel. We sketched both.



M1 sketch A Dashboard — all EDA views at once



M1 sketch B Story — the curation funnel as a hook

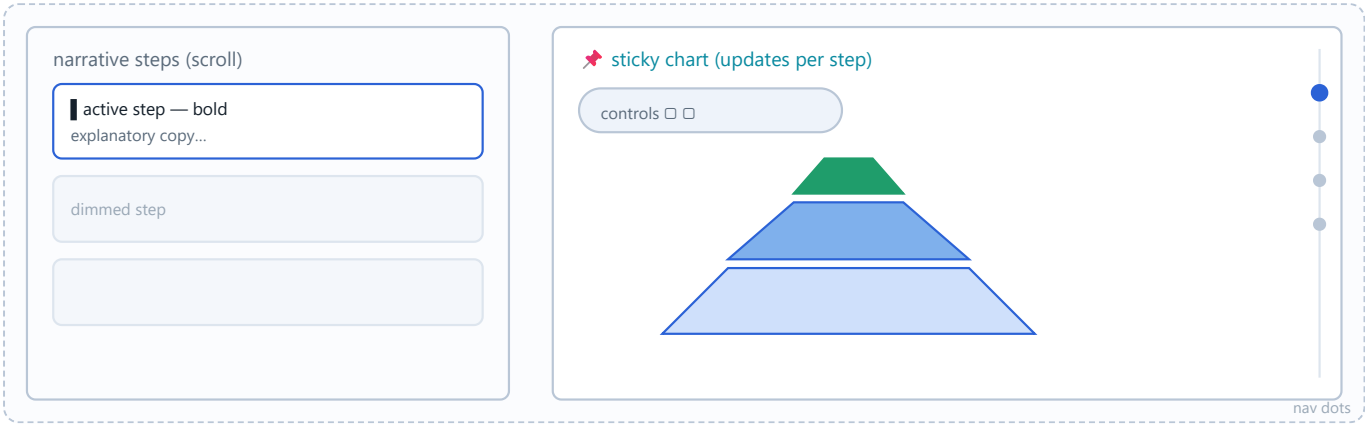
The decision

We picked **B, and folded A into it**. The funnel was the single most striking idea in the EDA — it turns an abstract statistic ("85% is discarded") into a physical shape. But we did not want to lose the exploratory richness of the dashboard, so each scrolly section embeds the relevant dashboard view as an *interactive*, controllable chart. The reader is guided, but never trapped.

Milestone-1 plan	What changed for Milestone 3	Why
Static dashboard of 24 EDA figures	7 curated, linked scenes in a scroll narrative	24 plots overwhelm a lay reader; a story sequences the insights
Funnel as a single static figure	Funnel that reveals stage-by-stage on scroll, with hover detail	Reveal = drama; hover = depth without clutter
Top-domains bar chart	Force-packed "Domain Galaxy" with search + category filter + size toggle	A bar chart of 126 domains is dull; a galaxy invites exploration
Two separate concentration plots	One view toggling Lorenz ↔ token-coverage, with hover read-out	They answer the same question; toggling links them
Light Matplotlib aesthetic	Dark, editorial theme with one shared palette	Dark canvas suits an "inside the machine" mood and makes colour pop

04 The final information architecture

The final site is one continuous scroll. A pinned graphic on the right updates as narrative "steps" pass the centre of the viewport on the left — the canonical scrollytelling pattern. We sketched the layout before building it:



M3 final layout Sticky graphic + scrolling steps, progress bar & section dots

The seven scenes

#	Scene	Chart type	Interaction beyond scroll
1	The Curation Funnel	Reveal funnel	Hover any stage → what it removes & %
2	Document Length	Histogram	Linear ↔ log toggle; hover bins
3	Language Score	Histogram	Threshold + tail highlighting; hover
4	The Web Through Time	Bar timeline	Metric & year/snapshot toggles; hover
5	The Domain Galaxy	Force bubbles	Search, category filter, size-by toggle
6	Concentration	Lorenz / coverage	Mode toggle; hover read-out of any share
7	The Quality Landscape	2-D heatmap	Cell highlighting; hover counts

Each scene is an independent ES module returning `{ update(step), resize() }`; the orchestrator maps a `data-chart` attribute to the module and forwards scroll events. This made the story re-orderable late in the project without touching chart internals.

05 Visual design decisions

A dark, editorial canvas

We moved away from the Milestone-1 Matplotlib palette to a dark theme. The mood we wanted was "inside the machine"; dark backgrounds also make the accent colours luminous and reduce visual fatigue across a long scroll. A single palette is shared between the CSS variables and the D3 config so nothing drifts.

Type & motion

Inter for UI, JetBrains Mono for figures. Motion is purposeful: transitions only fire on a state change (a new step, a toggle), never as idle decoration, so the reader's attention is guided rather than nagged.

Encoding choices

- **Funnel width = token volume.** A length encoding everyone reads instantly; the taper *is* the message.
- **Galaxy: area = volume, colour = category.** Area for magnitude, hue for a categorical attribute — two channels, no overload.
- **Log toggle** on the length histogram: the linear view shows "most pages are tiny", the log view reveals the log-normal shape. Letting the reader flip it themselves teaches the idea.
- **Sequential heat ramp** on the quality grid, on a `√` scale so the dominant cell does not wash out everything else.

Restraint

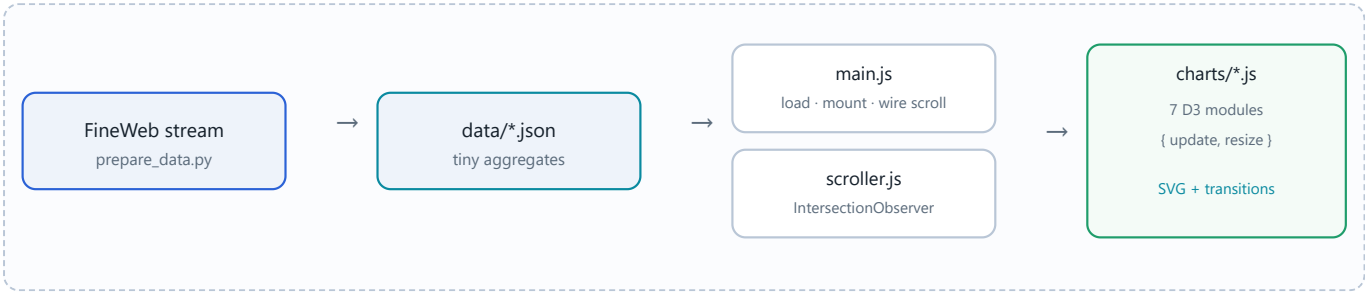
We deliberately cut a word-cloud and a chars-per-token panel from the EDA: both were visually busy and added little to the argument. Editing *out* was as important as designing in.

06 Making it interactive & legible

Three rules kept the interactivity honest. **(1) Scroll sets a sensible default; controls let you override** — e.g. the timeline opens on "documents per year" but you can switch to tokens or to all 96 snapshots. **(2) Hover always explains**, with a single shared tooltip that re-positions to stay on-screen. **(3) Nothing is a dead end** — the domain search, legend filters, and toggles reward curiosity after the guided beat has played.

07 Technical implementation

The site is intentionally **build-free**: vanilla HTML, ES modules, and D3 v7 from a CDN. There is no bundler and no `node_modules`, which means it deploys to GitHub Pages as-is and a teammate can run it with one `python -m http.server` command. The architecture is a thin pipeline:



Data pipeline → static JSON → orchestrator + scroll controller → seven chart modules

Data, two ways

The charts consume **aggregates**, not raw text, so `data/` is ~50 KB and loads instantly. `prepare_data.py` streams the real `sample-10BT` subset and exports the aggregates; `generate_aggregates.py` reproduces the same schema from a sample whose distributions are faithful to the Milestone-1 EDA, so the repo is runnable offline. Both write identical filenames.

Why no scroll library

A ~40-line `IntersectionObserver` controller replaces a dependency like Scrollama. A 0-height trigger band at the viewport centre guarantees exactly one active step, which is all the story needs. Fewer dependencies, easier to reason about, nothing to keep up to date.

`d3 v7` `ES modules` `IntersectionObserver` `ResizeObserver` `CSS sticky/grid` `no build step`

08 Challenges & how we solved them

1 · Ten billion tokens will not fit in a browser

The dataset is far too large to ship. We resolved this by separating *analysis* from *presentation*: all heavy lifting happens once in Python and is distilled into small JSON aggregates the browser can load in milliseconds. This also made the site reproducible — re-run one script to refresh the data.

2 · A self-contained, offline-runnable repository

Streaming the live dataset needs network, the `datasets` library, and several minutes. To keep the repo runnable instantly (for graders and teammates), we ship pre-computed aggregates generated from a sample that is statistically faithful to the EDA, and document exactly which parts are faithful versus illustrative. The real pipeline remains one command away.

3 · A stable force-directed galaxy

Our first bubble layout used a positive many-body force and collapsed to a single point; a negative one fought the centering and drifted off-screen. The fix was to drop charge entirely and rely on centering forces plus collision, with position clamping on every tick — a calm, packed cluster that re-settles smoothly when you re-size by tokens.

4 · Honest histograms

Clipping token counts to a maximum piled the entire long tail into the final bin, inventing a spike that isn't real. We switched to truncating the view (letting the histogram ignore the overflow) so the linear chart is faithful, and reserved the heavy tail for the explicit log-scale view.

5 · Responsiveness without a framework

Each chart exposes `resize()` and re-derives its scales from the live container size; a debounced `ResizeObserver` /window handler drives it. On narrow screens the CSS grid collapses so the graphic pins above the narrative instead of beside it.

09 What we learned & what is next

Lessons

- **Editing is design.** The story got stronger every time we removed a chart, not added one.
- **Aggregate early.** Deciding the JSON schema up front let the front-end and data work proceed in parallel.
- **Defaults teach, controls reward.** Scroll-set defaults carry the lay reader; the controls satisfy the expert.
- **Faithfulness must be stated.** Being explicit about faithful-vs-illustrative data is part of visual integrity.

Future work

- Stream a larger sample and surface real per-domain documents (a "peek at a page" panel).
- A small-multiples comparison of two crawl years side by side.
- Keyboard-only walkthrough mode and reduced-motion preference support.
- Server-side pre-rendering of the first scene for an instant first paint.

10 Tools & resources used

Purpose	Tool / resource
Visualization	D3.js v7 (selections, scales, forces, transitions, axes)
Data processing	Python · pandas · NumPy · HuggingFace `datasets` · `tldextract`
Scrollytelling	Native `IntersectionObserver` (custom controller)
Hosting	GitHub Pages (static)
Dataset & paper	FineWeb (HuggingFace) · Penedo et al., *The FineWeb Datasets*, NeurIPS 2024
Fonts	Inter · JetBrains Mono · Lora (this report)

All visualization code is our own. We consulted the D3 documentation and the FineWeb dataset card and paper for the curation-pipeline figures. Generative-AI assistance, where used, was limited to boilerplate and is reviewed and owned by the team.

11 Contribution breakdown

This is an **individual submission** by **Alireza Abdollahpoorrostam** (SCIPER 380830). The table below maps the work to the grading rubric; all parts were completed by the sole author.

Component	Work completed	Share
Data & pipeline	Milestone-1 EDA; streaming export (`prepare_data.py`) and the faithful aggregate generator (`generate_aggregates.py`); JSON aggregate schema.	100%
Visualization & engineering	Build-free front-end (`main.js` , `scroller.js`) and all seven D3 chart modules; interactions, transitions, responsiveness.	100%
Design & narrative	Visual design system and palette; story structure and copywriting; the dark editorial theme.	100%
Process book & screencast	This report and the screencast script & storyboard.	100%

As an individual submission, all decisions on scope, encodings, and the narrative order were made by the sole author.

12 Links

Live visualization	`https://com-480-data-visualization.github.io/FineWeb_Alireza/`
GitHub repository	`https://github.com/com-480-data-visualization/FineWeb_Alireza`
Screencast (≤ 2 min)	*add your video link before submitting*
Milestone-1 EDA	`FineWeb_EDA.ipynb`